

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY

UNIVERSITY OF SCIENCE

FACULTY OF INFORMATION TECHNOLOGY

FACTORY METHOD REPORT

CS202

Group members:

1. 25125030 - Võ Gia Minh
2. 25125055 - Phạm Huy Khánh
3. 25125066 - Nguyễn Quốc Thịnh
4. 25125085 - Chu Nguyễn Gia Khánh

21st August 2026



Contents

1	A Real-World Problem	2
2	A Naive Solution	2
3	Problems with the Naive Solution	4
4	Introduction to the Factory Method Pattern	5
5	Implementation	6
5.1	The general class diagram for the factory method design pattern	6
5.2	The factory method implementation of the problem raised at the beginning section .	7
6	Pros and Cons of the Factory Method Pattern	7
7	Other Real-World Problems and Applications in Web and Mobile Development	8
8	Quiz	8
9	References	13
10	Appendices	14

1 A Real-World Problem

Consider the architecture of a logistics and transportation management system. Initially, the software is designed exclusively for a localized delivery company, meaning the primary mode of transportation is by road. Consequently, the core business logic is heavily integrated with a **Truck** class.

As the enterprise expands its operations globally, a new requirement emerges to incorporate maritime logistics, necessitating the introduction of a **Ship** class. Furthermore, future expansions might require air freight (**Airplane**) or rail (**Train**). The fundamental problem is how to design the system so that it can dynamically construct different modes of transportation (Truck, Ship, etc.) without requiring the core logistics framework to know the exact concrete classes being instantiated.

2 A Naive Solution

A typical naive approach is to handle object instantiation directly within the client code (the business logic). This is usually achieved by passing a flag or type identifier and using conditional logic (**if-else** or **switch** statements) to determine which specific object to allocate.

```
#include <iostream>
#include <string>

// Abstract base class
class Transport {
public:
    virtual void deliver() = 0;
    virtual ~Transport() = default;
};

// Concrete class 1
class Truck : public Transport {
public:
    void deliver() override {
```

```
        std::cout << "Delivering cargo by land in a box.\n";
    }
};

// Concrete class 2
class Ship : public Transport {
public:
    void deliver() override {
        std::cout << "Delivering cargo by sea in a container.\n";
    }
};

// Client Code (The Naive Logistics Application)
class LogisticsApp {
public:
    void planDelivery(const std::string& transportType) {
        Transport* transport = nullptr;

        // Naive instantiation: tightly coupled conditional logic
        if (transportType == "Road") {
            transport = new Truck();
        } else if (transportType == "Sea") {
            transport = new Ship();
        } else {
            std::cout << "Unknown transport type.\n";
            return;
        }

        // Core business logic
        transport->deliver();
    }
};
```

```
        delete transport;
    }
};

int main() {
    LogisticsApp app;
    app.planDelivery("Road");
    app.planDelivery("Sea");
    return 0;
}
```

3 Problems with the Naive Solution

Implementing object creation in this manner introduces several architectural vulnerabilities:

- **Violation of the Open/Closed Principle (OCP):** The software is not closed for modification. Every time a new transportation method (e.g., **Airplane**) is introduced, the developer must open the **LogisticsApp** class and modify the existing **if-else** block. In a large-scale application, modifying core logic to add minor extensions is highly error-prone.
- **High Coupling:** The **LogisticsApp** (client code) is tightly coupled to the concrete classes (**Truck** and **Ship**). It must import and possess explicit knowledge of every specific implementation, rather than communicating strictly through abstract interfaces.
- **Code Duplication and Maintenance Overhead:** If the instantiation logic requires complex setup parameters (e.g., initializing a **Truck** requires assigning a driver, a license plate, and fuel capacity), this initialization code must be duplicated everywhere a **Truck** is created. If the initialization process changes in the future, developers must hunt down and update every conditional instantiation block across the codebase.

4 Introduction to the Factory Method Pattern

The Factory Method is a creational design pattern that separates object creation from object usage. Instead of letting client code directly instantiate concrete classes, the pattern defines a common method, called a "factory method", for creating objects. The concrete creation logic is hidden inside subclasses, which override the factory method to decide which concrete product class should be instantiated.

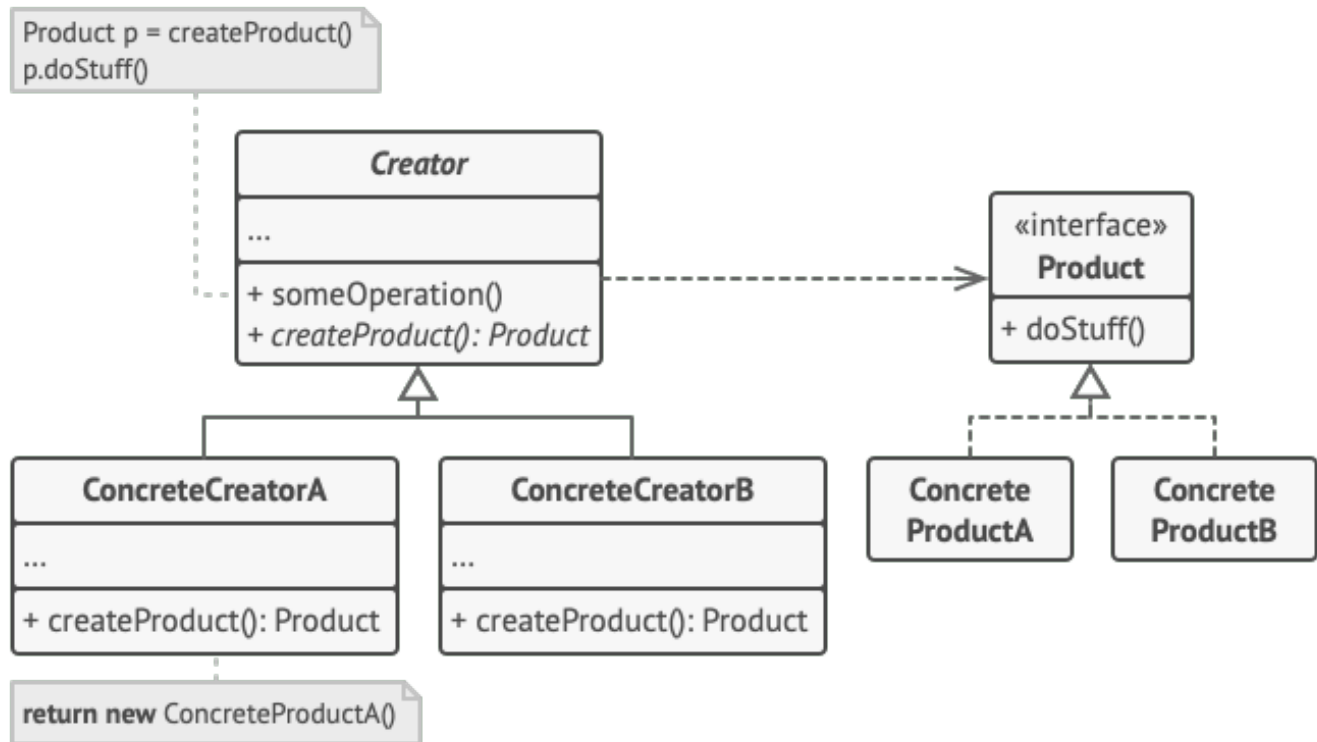
In the naive design, the client code contains object creation logic directly inside its business logic. It uses if-else to decide which class to instantiate. This causes the client code to know too much about concrete classes, making the system harder to extend and maintain. Factory Method improves this design by moving those creation decisions into subclasses, allowing the main workflow to depend on abstractions instead of concrete implementations.

The Factory Method contains four main roles:

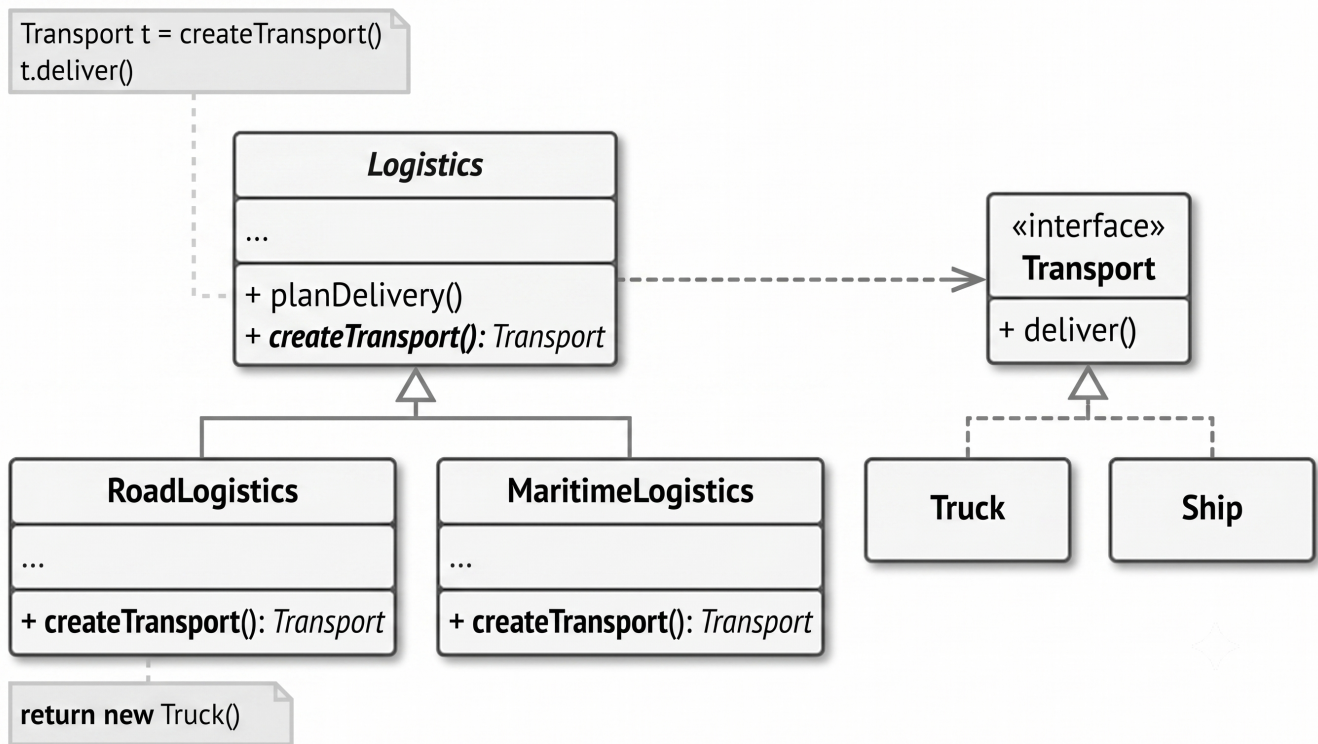
- **Product:** The interface or abstract class of the objects being created. The client or main workflow uses this abstraction instead of depending directly on concrete product classes. For example, `Transport` instead of `Truck` or `Ship`.
- **ConcreteProduct:** The specific classes that implement the `Product` interface, such as `Truck` and `Ship`. These are the actual objects created by the factory method.
- **Creator:** The abstract class or base class that declares the factory method. It may also contain the main workflow that uses `Product` objects.
- **ConcreteCreator:** The subclasses that override the factory method. Each `ConcreteCreator` decides which `ConcreteProduct` to instantiate. This is where the concrete creation logic is hidden.

5 Implementation

5.1 The general class diagram for the factory method design pattern



5.2 The factory method implementation of the problem raised at the beginning section



[Click here to see the source code for the problem.](#)

6 Pros and Cons of the Factory Method Pattern

The Factory Method Pattern provides several benefits, especially for systems that need to support multiple related object types and future extensions. However, it also introduces additional structure, so it should be used only when the added flexibility is worth the complexity.

Pros:

- **Reduces coupling:** The client code depends on an abstract **Product** interface instead of depending directly on concrete classes. This makes the code more flexible and easier to change.
- **Single Responsibility Principle:** The client code focuses on using objects, while the factory method handles the creation of concrete products. This improves separation of responsibilities.
- **Open/Closed Principle:** New product types can be added by creating new **ConcreteProduct** and **ConcreteCreator** classes, without modifying the existing core

workflow.

- **Improves maintainability:** Since the creation logic is isolated inside creator subclasses, changes to how an object is created are easier to locate and update.

Cons:

- **More classes:** Each new product type may require a corresponding creator class. This can make the project larger and harder to navigate.
- **Complexity:** For small programs with only a few fixed object types, Factory Method may be unnecessary and may make the design harder to understand.

7 Other Real-World Problems and Applications in Web and Mobile Development

The Factory Method Pattern can be used in many systems where different types of objects need to be created, but the main logic should not depend directly on concrete classes.

- **Authentication system:** A web application may support different login methods, such as email login, Google login, or Facebook login. The main login workflow can depend on a common `AuthProvider` interface, while factory methods create concrete classes such as `EmailAuthProvider`, `GoogleAuthProvider`, or `FacebookAuthProvider`.
- **Mobile development:** A mobile application may need different UI components or services for Android and iOS. For example, factory methods can create `AndroidButton` or `IOSButton`, while the main UI code works with a general `Button` interface.

Overall, Factory Method is useful when object creation varies, but the way the objects are used is similar.

8 Quiz

Question 1: What is the primary purpose of the Factory Design Pattern?

- A. To create a single instance of a class.
- B. To encapsulate object creation and hide the instantiation logic from the client.
- C. To make all classes inherit from a common superclass.
- D. To improve algorithm efficiency.

Answer: B

Explanation: The Factory Pattern separates object creation from object usage, allowing clients to depend on abstractions instead of concrete classes.

Question 2: Identify the Pattern

Consider the following code:

```
1 class Shape {
2 public:
3     virtual void draw() = 0;
4 };
5
6 class Circle : public Shape {
7 public:
8     void draw() override {
9         cout << "Circle\n";
10    }
11 };
12
13 class Square : public Shape {
14 public:
15     void draw() override {
16         cout << "Square\n";
17    }
18 };
19
20 Shape* createShape(string type) {
21     if (type == "circle")
22         return new Circle();
23     else if (type == "square")
```

```
24     return new Square();  
25     return nullptr;  
26 }
```

Why is createShape() considered a factory?

- A. It contains inheritance.
- B. It creates objects without exposing the concrete classes to the client.
- C. It uses polymorphism.
- D. It has virtual functions.

Answer: B

Question 3: Complete the sentence.

The Factory Pattern follows the principle of programming to _____ rather than concrete implementations.

Answer: interfaces (or abstractions)

Quiz 4: Code Analysis

Suppose you have:

```
1 Vehicle* vehicle = Factory::createVehicle("Car");  
2 vehicle->move();
```

Later, you add:

```
1 class Bike : public Vehicle { ... };
```

Which code should ideally change?

- A. Client code
- B. Vehicle interface
- C. Factory
- D. All classes

Answer: C

Explanation: Only the factory should know how to instantiate the new class.

Quiz 5: True or False

- A. The Factory Pattern reduces coupling between clients and concrete classes.
- B. The client must know which constructor each concrete product uses.
- C. The Factory Pattern usually works together with polymorphism.
- D. Factory Pattern eliminates inheritance completely.

Answer:

- A. True
- B. False
- C. True
- D. False

Quiz 6: Find the Problem

Consider:

```
1 if (type == "Circle")
2     return new Circle();
3 else if (type == "Square")
4     return new Square();
5 else if (type == "Triangle")
6     return new Triangle();
7 else if (type == "Rectangle")
8     return new Rectangle();
9 ...
```

What disadvantage does this factory have?

- A. Too many constructors
- B. Violates the Open/Closed Principle because every new product requires modifying the factory

- C. No inheritance
- D. Too much polymorphism

Answer: B

Quiz 7: Design question

You are designing a game.

Characters include:

- Knight
- Archer
- Wizard

Each has an attack() function.

Which implementation best follows the Factory Pattern?

A.

```
Knight k;
```

B.

```
if (type == "Knight")  
    return new Knight();
```

In client code

C.

```
Character* c = CharacterFactory::create(type);
```

D.

```
switch (type)  
{  
    case 1: Knight...  
}
```

Answer: C **Explanation:** Because object creation is centralized inside the factory, keeping the client independent of concrete character classes.

Quiz 8: Which of the following scenarios is the best candidate for using the Factory Design Pattern?

- A. A program that always creates exactly one `Logger` object.
- B. A drawing application that needs to create different shapes (`Circle`, `Rectangle`, `Triangle`) based on user input.
- C. A mathematical function that calculates the factorial of a number.
- D. A class that only stores employee information.

Answer: B

Explanation:

The Factory Pattern is useful when:

- The exact type of object to create is determined at runtime.
- The client should not depend on concrete classes.
- New object types may be added in the future with minimal changes to client code.

A drawing application is a classic example because the factory can create different Shape objects depending on the user's choice.

9 References

1. Refactoring.Guru, "Factory Method," *Design Patterns*. Available: <https://refactoring.guru/design-patterns/factory-method>.
2. SourceMaking, "Factory Method Design Pattern." Available: https://sourcemaking.com/design_patterns/factory_method.
3. Viblo, "Tổng quan về Factory Method Design Pattern". Available: <https://viblo.asia/p/factory-method-design-pattern-tro-thu-dac-luc-cua-developers-924lJBLYlPM>.

10 Appendices